

Title Page

Title: Inter-process TinyFile Communication & Compression Service

Author: Barnabe Marty, Padraig Littlefield

Date: October 15, 2025

Abstract

This report describes the design and implementation of a shared memory-based file compression service that applies inter-process communication via a client-server like architecture. The TinyFile service acts as a daemon process that compresses files, upon request, from multiple clients simultaneously, utilizing the POSIX shared memory segments and synchronized request/response queues for data exchange.

The system was designed for both synchronous (blocking) and asynchronous (non-blocking) communication, allowing clients to wait for compression or continue executing while the service completes their request in the background. Some key design features that were made are: (1) the initialization and layout of the data structure for requests when the service is launched, (2) the setup of the metadata for clients before sending requests to the service, (3) the flow of information (i.e. information flow for IPC) between service and client, and (4) the cleanup process of both the client and service when the processes are done executing or are terminated.

Table of Contents

Title Page	1
Abstract	1
System Design & Implementation	2
Architecture Overview	2
Shared Memory	2
Communication Protocol	3
SYNC and ASYNC Implementation	5
Sample Application	6
Experiment	7
Setup	7
Results	8
Analysis	10

System Design & Implementation

Architecture Overview

The TinyFile system consists of three major components: a daemon service that manages shared memory and processes requests for compression (TinyFile service), a client library providing APIs to request compression from the TinyFile service, and a sample application demonstrating full functionality. The inter-process communication occurs all through shared memory segments, eliminating slower IPC mechanisms. Figure 1 displays the architecture of the full system with clients accessing shared memory monitored by the service.

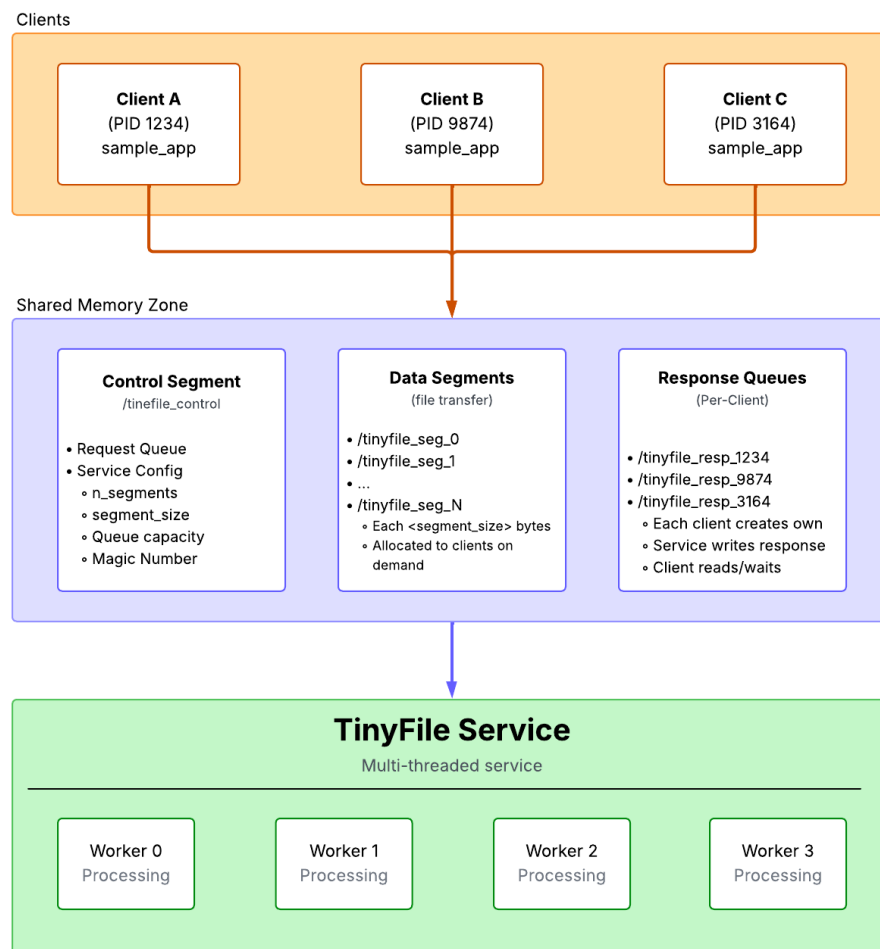


Figure 1: High Level System Block Diagram of TinyFile System

Shared Memory

The system sets up three shared memory blocks. The **control segment** (/tinyfile_control) contains service configuration information (segment count, segment

sizes, magic number¹...) and the service request queue for clients. This provides service discovery and a centralized way to handle requests. **Data segments** (`/tinyfile_seg_0` through `/tinyfile_seg_N`) make a pool of fixed size buffers for data transfer. The amount of data segments and their size is determined when launching the TinyFile service using command line arguments.

Clients use data segments by requesting allocation of said segment via the control queue, receiving a segment identification number, and using the data segment to transfer file data over to the service. Multiple segments allow multiple clients to submit requests for compression. **Response queues** (`/tinyfile_resp_<PID>`) are per-client queues where the service delivers responses for any request the client makes.

Communication Protocol

Queues are initialized using circular buffers with mutexes and condition variables for metadata. The request queue for the service uses a producer-consumer design: clients enqueue and block when the queue is full, while workers dequeue and block when the queue is empty. The client then looks through the response queue to see how the service responded and whether it can proceed in the process. Queues have a fixed sized to allow for simple yet efficient implementation and will block enqueueing of entries until an element is removed. The request queue capacity is set to 32 for our design, and the each response queue can hold 10 entries (this was found to be the best for our experiments).

The compression protocol has five different request types: `REQ_ALLOC_SEGMENT` requests a data segment for the client to use as a way to exchange data with the serve and initializes server side state, `REQ_SENT_DATA` notifies the service that a data chunk of file is ready to copy into its own memory, `REQ_COMPRESS` triggers compression after all input data has been received², `REQ_REC_DATA` requests the next output packet of compression, and `REQ_FREE_SEGMENT` releases resources on server-side to indicate the full compression and data transfer is complete. Files larger than data segment size are transferred in packets/chunks of data. For example, a 10KB file with a service set to 4KB segments, three input packets are sent via shared data segment, the service accumulates data in an internal buffer, compresses it, and then streams the output back into however many packets necessary. Figure 2 shows the complete sequence of IPC and compression.

¹ The magic number is a way for the library to make sure that the control segment is not corrupt when connecting to it.

² The service will immediately start sending a compressed file chunk (if compression succeeds) to the client.

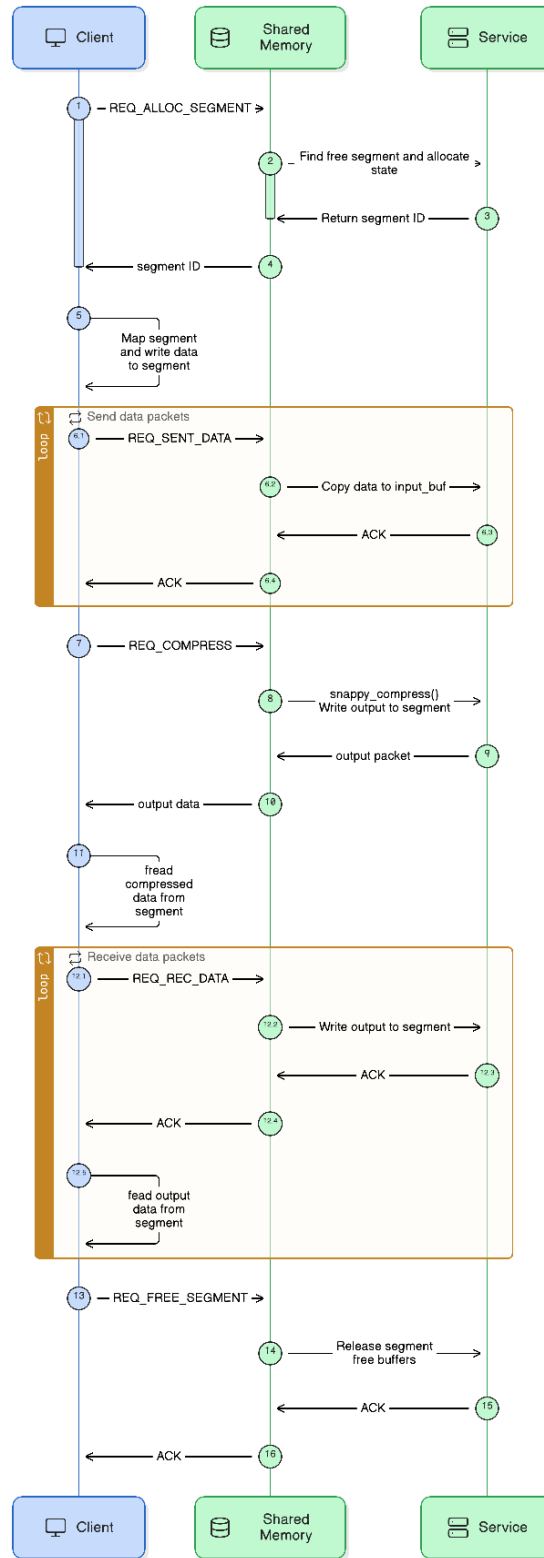


Figure 2: Full Overview of Compression/Communication Protocol of TinyFile System

SYNC and ASYNC Implementation

The client API has two modes for requests: synchronous (SYNC) or asynchronous (ASYNC) mode. These fundamentally go through the same communication protocol but allow the client to send a compression job in different ways.

In SYNC mode, `tinyfile_compress_sync()` executes the full communication scheme within the calling client, blocking at each step until the service responds. The function goes through the whole communication scheme while blocking at each request to look for a response. Client service time (CST) spans the entire operation. The application cannot perform other work during compression as this is a blocking call.

In ASYNC mode, `tinyfile_compress_async()` spawns a worker thread from main thread that detaches and calls the synchronous implementation internally. The library keeps track of all jobs via an array of active async jobs with a maximum capacity of 16 concurrent jobs per client. The function immediately returns a job ID to the main thread, allowing the application to continue execution. Then compression finishes, the background thread goes to a user-provided callback function with the result status and CST, then removes itself from the tracking array³. The client API library also provides `tinyfile_wait_all_async()` to block until all background jobs finish (if the main client thread ever needs to or to ensure proper cleanup of resources). Figure 3 contrasts the two modes.

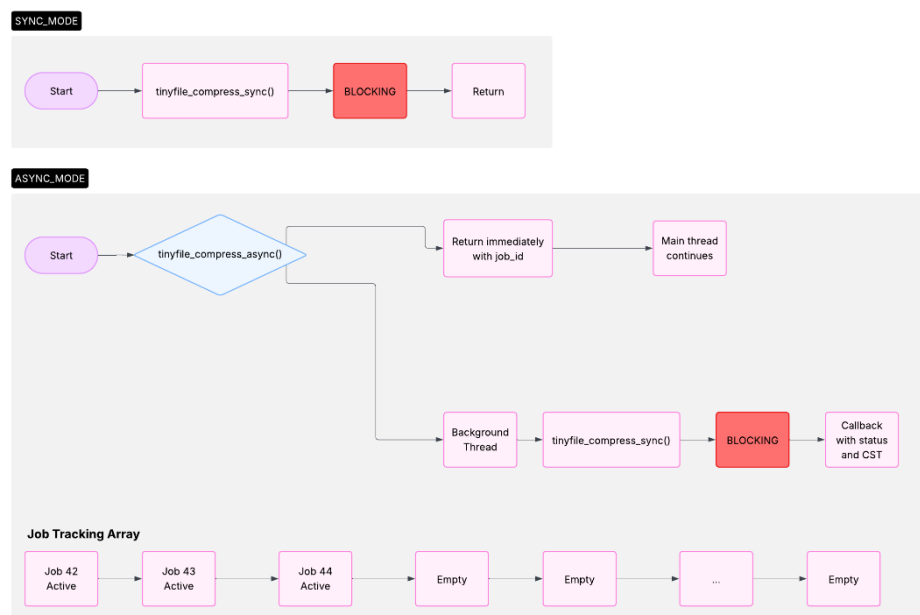


Figure 3: Block Diagram describing high-level overview of SYNC vs. ASYNC request

³ Acquiring the CST is implemented in the sample application, while removing the thread from the tracking array is implemented in the client API library.

Sample Application

The sample application helps express the library functionality through single-file and batch compression. For single file compression, SYNC mode blocks and prints CST upon full completion of a compression request via the client API library, while ASYNC mode returns immediately, simulates other work, and gets notified upon completion via a callback function. For batch mode with multiple files, SYNC processes files sequentially, while ASYNC launches all compression requests concurrently then waits for all to complete.

The application measures and logs CST for each compression, supporting the experiments described in the Experiments section. Command-line arguments control everything, with documentation included in the repository.

Experiment

To concretely analyze the performance of the TinyFile service and its client APIs, three experiments were conducted on the system to quantify performance with each experiment changing different parameters.

Setup

Experiment 1: Varying Segment Size: This experiment tested how different shared memory segment sizes affect the CST. The sample application repeatedly compressed all five of the test files (Huge.jpg, Large.txt, Medium.pdf, Small.jpg, Tiny.txt) using consecutive synchronous calls to the service. The service was restarted with different segment sizes (32, 64, 128, 256, 512, 1024, 2048, 4096, and 8192 bytes) while keeping the number of segments fixed at 5. The average CST from compressing the five files were recorded for each segment size.

Experiment 2: Varying Number of Segments: This experiment tested how the number of shared memory segments affects CST under concurrent load. Ten clients were instantiated at once, each running the aforementioned sample application. The service was restarted with different segment counts (1, 3, and 5) while keeping the segment size fixed at 1024 bytes. The average CST from compressing the five files across the ten clients was recorded for each segment count.

Experiment 3: Asynchronous Communication: This experiment tested how the CST was affected by making synchronous calls to the TinyFile service versus making asynchronous calls. Experiment 1 and Experiment 2 were reran using the ASYNC state of our sample application, which compresses the five test files using asynchronous calls to the service.

Results

Below are all figures to help visualize experimental data that was gathered from the described experiments above. Each detail critical performance benchmarks of the designed TinyFile service and TinyFile client API under with varying parameters.

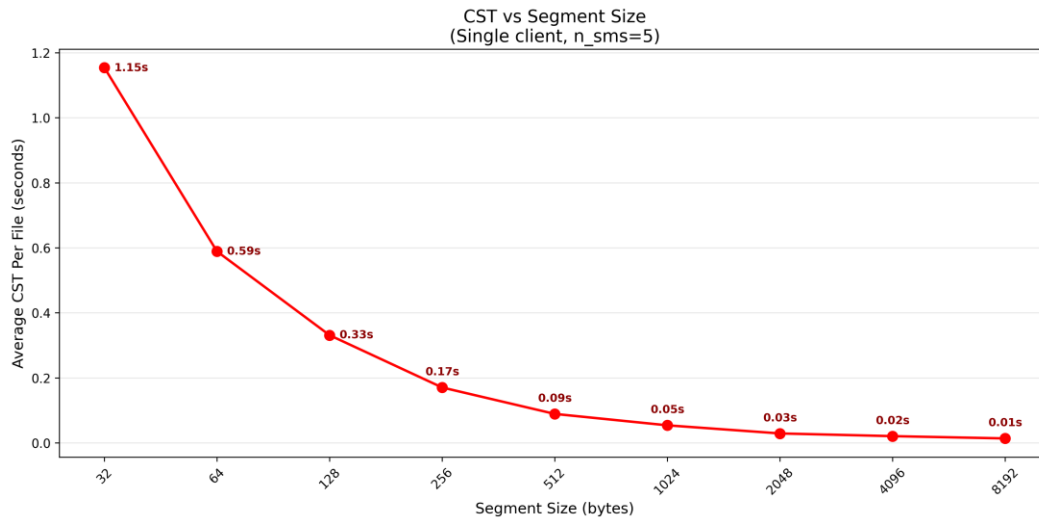


Figure 4: Experimental data showing CST vs. Segment Size of Shared Memory

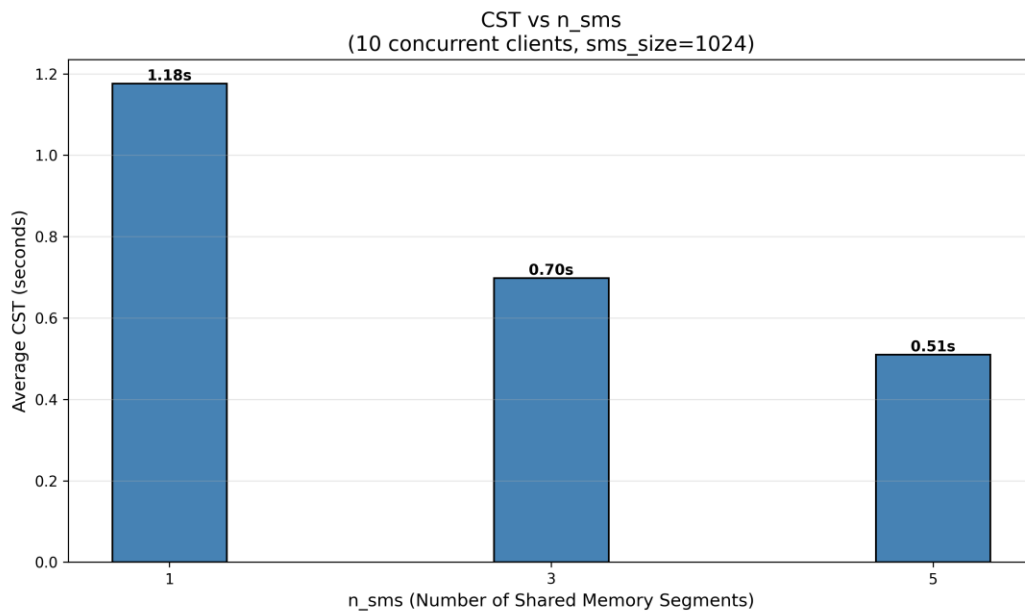


Figure 5: Experimental data showing CST vs. Number of Segments available in TinyFile Service

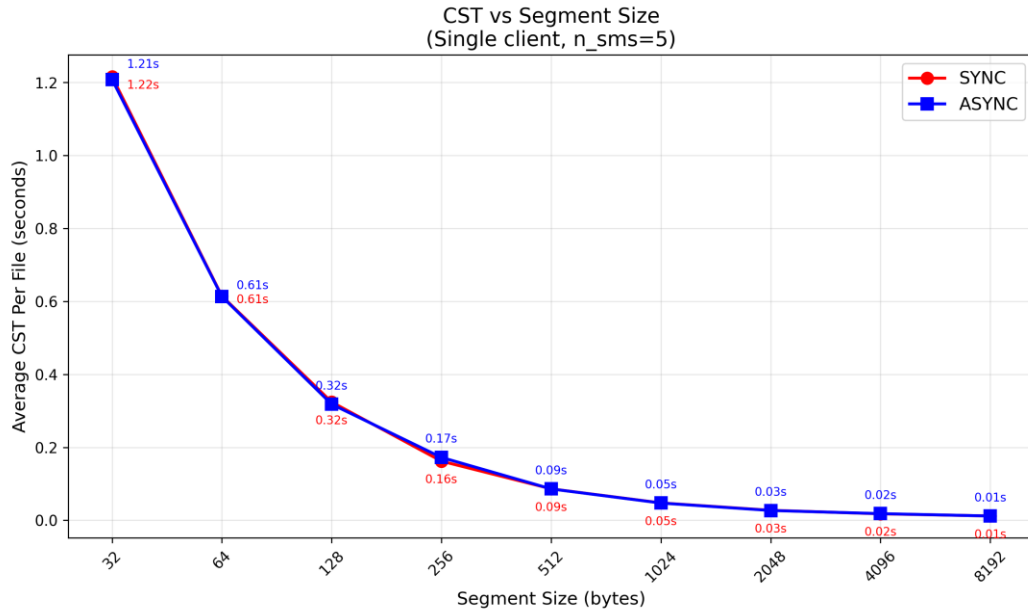


Figure 6: Experimental data benchmarking performance between SYNC vs. ASYNC for varying segment sizes

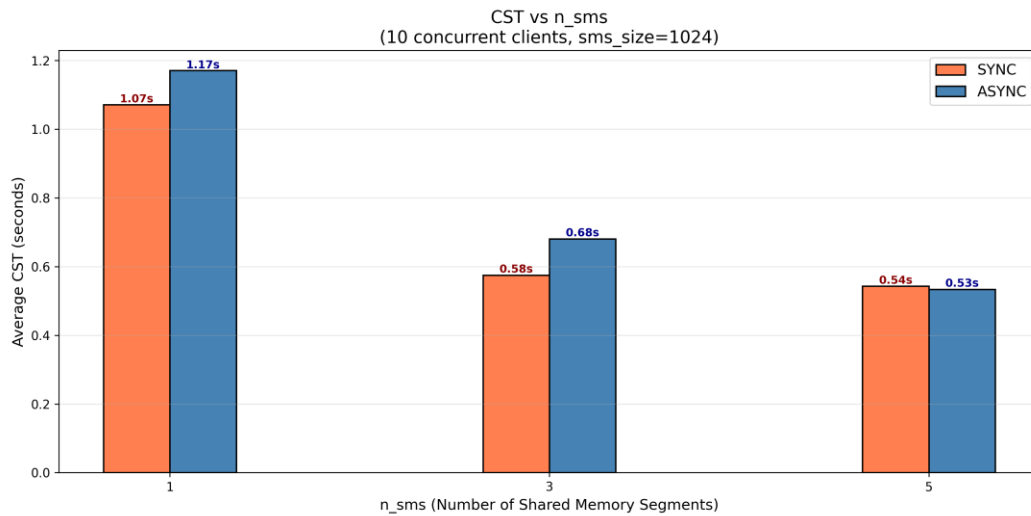


Figure 7: Experimental data benchmarking performance between SYNC vs. ASYNC for varying n_sms

Analysis

Experiment 1: Varying Segment Size: The results from Experiment 1 (Figure 4) demonstrate a clear inverse relationship between segment size and CST, with the CST decaying exponentially as the segment size is increased. The largest performance gains can be found from increasing the segment size from 32 bytes to 256 bytes, with the average CST decreasing by 85.5%

The performance gains become less pronounced at larger segment sizes, exhibiting a logarithmic decay pattern. Segment sizes of 1024, 2048, 4096, and 8192 bytes yielded an average CST per file of 0.05, 0.03, 0.02, and 0.01 seconds respectively. This diminishing return occurs because the overhead of IPC becomes negligible, and the actual time spent compressing the file starts to dominate.

The poor performance at small segment sizes can be attributed to excessive amount of IPC required and synchronization overhead that must be incurred. With 32-byte segments, even the smallest test file (Tiny.txt) requires multiple segment transfers, each involving a request and response pair to and from the server. Larger segments allow entire small files or substantial portions of larger files to be transferred in a single operation, dramatically reducing the amount of expensive IPC required.

Experiment 2: Varying Number of Segments: The results from Experiment 2 (Figure 5) reveal how the segment count can determine the concurrent performance of the system. With only 1 segment available and 10 concurrent clients, the average CST was 1.13 seconds per file. This occurs because all clients must compete for the single shared memory segment, which forces a serialization bottleneck on what should be parallel and independent operations.

Increasing to 3 segments reduced average CST by 38% (from 1.07 to 0.70 seconds). This highlights how even slightly increasing the number of segments can dramatically improve concurrency by allowing multiple clients to simultaneously access the service. The most dramatic improvement came with 5 segments, which achieved a CST of 0.51 seconds, representing a 55% reduction from the single-segment case.

Experiment 3: Asynchronous Communication: Figures 6 and 7 demonstrate the performance characteristics of asynchronous communication compared to synchronous blocking calls across both experimental conditions.

In the single-client set-up from Experiment 1 (Figure 6), asynchronous communication achieved nearly identical performance to the synchronous version, with variations of less than 1% across all segment sizes. This shows that the asynchronous implementation introduces little overhead while providing the benefit of non-blocking operation.

In the concurrent ten-client scenario from Experiment 2 (Figure 7), the asynchronous version exhibited slightly degraded performance compared to synchronous calls. This performance penalty likely stems from increased contention, as all clients attempt to process their files in parallel rather than indirectly serializing their requests through blocking calls. When multiple asynchronous clients flood the service simultaneously, they compete more aggressively for the limited shared memory segments, potentially increasing wait times.

Despite this minor degradation under high concurrency, the asynchronous API demonstrates strong practical value. The performance difference is minimal, while providing the critical advantage that asynchronous callers can continue productive work instead of blocking idle, potentially achieving better overall system performance than the raw CST measurements suggest.